



Universidad Nacional Autónoma de México

Práctica 5 Compiladores 2027 - 1

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante de Laboratorio: Pedro Yamil Salazar González

Alumnos :

Marcial Ahedo Andrick Iorel

Guzmán Ramirez Diego Arturo

Equipo 06

Fecha de entrega : 23 / 10 / 2026

Objetivo

En la Práctica 4 ya podíamos revisar si un programa estaba bien escrito sintácticamente y construir su Árbol de Sintaxis Abstracta (AST), pero todavía no verificábamos si las variables que se utilizaban habían sido declaradas.
Para esta práctica implementamos una tabla de símbolos que nos permite guardar información de las variables, como su nombre, tipo y posición donde fueron declaradas.
También agregamos el manejo de ámbitos para comprobar en qué partes del programa se pueden utilizar esas variables.
Así podemos detectar variables no declaradas, redeclaraciones dentro de un mismo ámbito y permitir el shadowing, que ocurre cuando una variable tiene el mismo nombre que otra, pero se encuentra en un ámbito diferente.

Evolución respecto de la Práctica 4

La Práctica 5 se desarrolló utilizando el código que ya teníamos de la Práctica 4.
Conservamos el lexer, el parser descendente recursivo y el AST, no fue necesario volver a implementar esas partes.
El cambio principal fue agregar el análisis semántico después de construir el AST. Ahora el compilador no solo revisa que las instrucciones cumplan con la gramática de MiniC, también comprueba que las variables utilizadas tengan una declaración válida.

- Se agregó el módulo **semantic**, que contiene los archivos s**ymbol, scope, symbol_table y semantic.** Estos archivos se encargan de guardar la información de las variables, manejar los ámbitos y realizar las comprobaciones semánticas.

- Se implementó una tabla de símbolos para registrar las variables conforme aparecen sus declaraciones en el programa. Con esta tabla podemos buscar identificadores, detectar redeclaraciones y comprobar si una variable se encuentra dentro de un ámbito donde puede utilizarse.

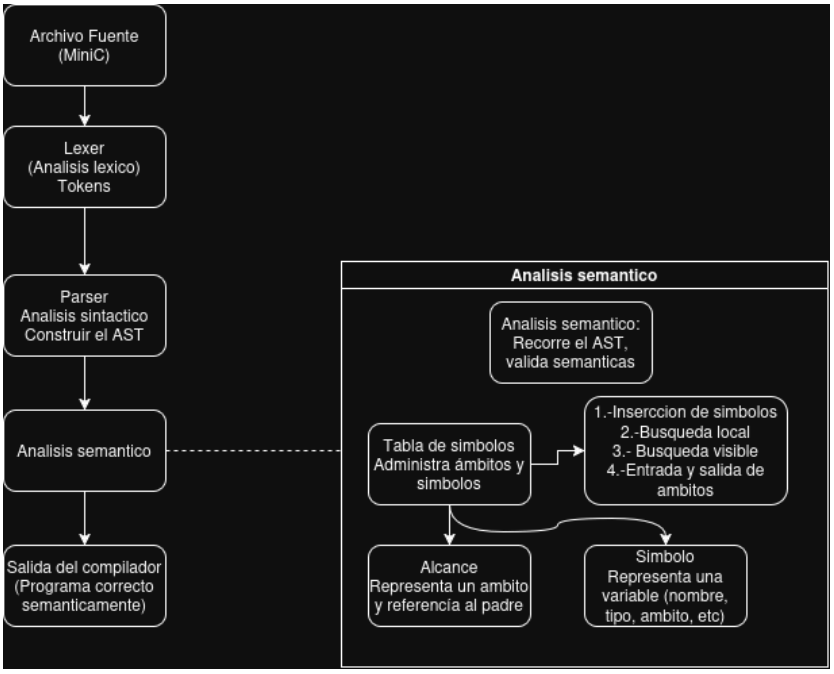
- Se agregó el recorrido del AST mediante **semantic_analyze**(). Esta función utiliza los nodos que construimos en la práctca 4 para revisar declaraciones, asignaciones, expresiones y estructuras como if y while. También permite continuar revisando el programa cuando encuentra errores semánticos, para detectar más de uno.

- Se modificó main.c para que el análisis semántico se ejecute después del parser, siempre que no existan errores léxicos o sintácticos. También cambiamos la salida del programa, ya que ahora solamente se imprime Programa semanticamente correcto. cuando todas las comprobaciones terminan bien.

- Se actualizó el Makefile para compilar los nuevos archivos y se agregaron pruebas para comprobar el funcionamiento de la tabla de símbolos y del analizador semántico.

Diseño del Analizador semántico

Para esta práctica agregamos un nuevo módulo que se encarga de revisar el AST y comprobar que las variables estén declaradas correctamente. Este módulo trabaja después del parser, por lo que recibe el árbol que ya fue construido y utiliza su información para realizar las validaciones.



Como podemos observar en el diagrama, primero el lexer reconoce los tokens del archivo, después el parser revisa que el programa cumpla con la gramática y construye el AST. Una vez que tenemos ese árbol, entra el analizador semántico para comprobar las declaraciones y los usos de variables.

Para organizar esta nueva parte utilizamos cuatro componentes principales:

Symbol(Símbolo): guarda la información de cada variable declarada, como su nombre, tipo, línea, columna y una referencia a su declaración dentro del AST.

Scope (Alcance) : representa un ámbito del programa. Cada ámbito guarda sus propias variables y tiene una referencia al ámbito padre, lo que permite acceder a variables declaradas en bloques exteriores.

SymbolTable(Tabla de símbolos): administra los símbolos y los ámbitos. Permite agregar variables, buscar declaraciones y entrar o salir de un ámbito cuando se recorren los bloques del programa.

SemanticAnalyzer(Análisis semántico): se encarga de recorrer el AST y utilizar la tabla de símbolos para comprobar las reglas semánticas. También registra los errores que encuentra durante el análisis.

Para guardar las variables de cada ámbito utilizamos arreglos dinámicos, que pueden aumentar su tamaño cuando se agregan más símbolos. Así almacenamos diferentes cantidades de variables sin establecer un límite fijo pequeño.
También decidimos conservar los ámbitos que se van creando hasta terminar el análisis. Cuando salimos de un bloque, sus variables dejan de estar disponibles para las búsquedas, pero la memoria se libera posteriormente cuando destruimos la tabla de símbolos.
La tabla de símbolos guarda el tipo de las variables, aunque todavía no lo utilizamos para comprobar si las operaciones o asignaciones son compatibles.

Tabla de símbolos y ámbitos

Para poder revisar las declaraciones de las variables implementamos una tabla de símbolos, que nos permite guardar información de los identificadores que aparecen en el programa. Así cuando encontramos una variable dentro de una asignación o expresión, podemos comprobar si ya fue declarada.

Para construir la tabla utilizamos tres estructuras principales: **Symbol, Scope y SymbolTable.**

La estructura Symbol representa una variable declarada. Guarda su nombre, el tipo (int o bool), el nivel de ámbito, la línea y columna donde fue declarada y una referencia al nodo del AST que corresponde a esa declaración. Para crear y liberar estos símbolos implementamos las funciones **symbol_create() y symbol_destroy().**

Scope representa un ámbito del programa. Cada ámbito contiene un arreglo dinámico donde se guardan los símbolos declarados directamente dentro de él. También tiene una referencia a su ámbito padre, que utilizamos cuando necesitamos buscar variables que fueron declaradas en bloques exteriores.

Los ámbitos se organizan por niveles. El programa comienza en el nivel 0, que corresponde al ámbito global. Cuando encontramos un bloque nuevo, se crea otro ámbito con un nivel mayor.

En este caso, **x** pertenece al ámbito global, **y** al primer bloque y **z** al bloque más interno. La variable **x** puede utilizarse dentro de los tres ámbitos, mientras que z solamente puede utilizarse dentro del bloque donde fue declarada.

```
int x;
{
   int y;
   {
      int z;
   }
}
```

SymbolTable se encarga de administrar estos ámbitos. Para ello implementamos funciones como **symbol_table_enter_scope() y symbol_table_exit_scope()**, que permiten entrar y salir de un ámbito mientras se analiza el programa.

También utilizamos **symbol_table_insert()** para registrar las variables y **symbol_table_destroy()** para liberar la memoria al terminar.
Para almacenar los símbolos utilizamos arreglos dinámicos que aumentan su capacidad mediante realloc cuando es necesario, por lo que no establecimos un límite fijo pequeño para la cantidad de variables.

Resolución de identificadores

Ya que tenemos la tabla de símbolos, necesitamos comprobar a qué declaración corresponde cada variable que aparece en el programa. Para esto implementamos dos tipos de búsqueda: la búsqueda local y la búsqueda visible.

La búsqueda local se realiza mediante **symbol_table_lookup_current().** Esta función revisa únicamente los símbolos del ámbito actual y la utilizamos principalmente para detectar redeclaraciones.

```
int x;
bool x;
```

La segunda declaración genera un error semántico porque x ya existe dentro del mismo ámbito. Cuando esto ocurre, conservamos la primera declaración y no la reemplazamos con la nueva.

La búsqueda visible se realiza mediante **symbol_table_lookup()**. A diferencia de la anterior, esta función comienza buscando en el ámbito actual y, si no encuentra la variable, continúa buscando en los ámbitos exteriores hasta llegar al global.
Esto también nos permite implementar el shadowing, que ocurre cuando declaramos una variable con el mismo nombre que otra, pero dentro de un ámbito diferente.

```
int x = 10;
{
   bool x = true;
   print(x);
}
print(x);
```

Este programa es válido porque las dos declaraciones de **x** pertenecen a ámbitos diferentes. Dentro del bloque se utiliza la variable de tipo bool, mientras que fuera del bloque se utiliza la variable global de tipo int.

Otra regla importante es que las variables deben declararse antes de utilizarse.También comprobamos que las variables dejen de ser visibles cuando termina su ámbito. Si declaramos una variable dentro de un bloque y después intentamos utilizarla fuera de él, el analizador reporta que no fue declarada.

Un caso especial es cuando una variable aparece dentro de su propia inicialización:

int x = x;

Primero registramos la declaración de x y después revisamos su inicializador. Todavía no verificamos si la variable tiene un valor inicializado.

Recorrido del AST

El análisis semántico se realiza recorriendo el AST que construimos en la Práctica 4. Para esto implementamos el archivo **semantic.c,** que contiene las funciones encargadas de revisar los diferentes tipos de nodos del árbol.

La función principal es **semantic_analyze(),** que recibe el nodo raíz del programa y comienza el recorrido. Después utiliza funciones auxiliares dependiendo del tipo de nodo que encuentra.
El recorrido comienza con **visit_program()**, que revisa las sentencias en el mismo orden en el que aparecen en el programa. Porque las variables deben declararse antes de utilizarse y no podemos registrar todas las declaraciones desde el principio.

Cuando encontramos una declaración, utilizamos **visit_declaration()**. Esta función comprueba si el nombre ya existe en el ámbito actual y, si no existe, crea el símbolo correspondiente y lo agrega a la tabla. Después revisa la expresión inicializadora, en caso de que tenga una.

Para las asignaciones utilizamos **visit_assignment(),** que primero comprueba si la variable a la que queremos asignar un valor está declarada. Después recorre la expresión de la asignación para verificar los demás identificadores que puedan aparecer.

int x;
x = y + 5;

En este caso, el analizador encuentra que x está declarada, pero cuando recorre la expresión detecta que y no tiene una declaración visible y genera un error semántico.

También implementamos **visit_expression()**, que permite revisar expresiones con operadores como suma, resta, multiplicación y operadores lógicos. Su función es recorrer

los operandos y comprobar los identificadores que aparecen, sin calcular el resultado de las operaciones.

Para los bloques utilizamos **visit_block().** Cuando el analizador encuentra un bloque, crea un ámbito nuevo, recorre sus sentencias y al terminar regresa al ámbito anterior. De esta manera las variables declaradas dentro del bloque dejan de estar disponibles fuera de él. En las sentencias if y while también revisamos las condiciones y sus cuerpos. Si contienen bloques, estos crean sus propios ámbitos (solamente revisamos que las variables utilizadas estén declaradas).

Cuando el recorrido encuentra un error semántico, intenta continuar con las demás sentencias para detectar otros errores dentro del mismo programa. Así reporta varios problemas en una sola ejecución, en lugar de detenernos después del primero.

Propiedad y memoria

En esta práctica seguimos utilizando memoria dinámica, la diferencia es que ahora también necesitamos reservar memoria para los símbolos, los ámbitos y los arreglos donde se guardan las variables declaradas. Para evitar problemas de memoria, cada módulo se encarga de liberar los recursos que le corresponden.

En el caso de Symbol, cuando creamos un símbolo con **symbol_create()**, reservamos memoria para su estructura y hacemos una copia del nombre de la variable. Esto permite que el símbolo conserve el nombre sin depender de otras cadenas que puedan liberarse durante el análisis. Para liberar esta memoria utilizamos **symbol_destroy()**, que elimina el nombre almacenado y después libera la estructura del símbolo.

Los ámbitos también utilizan memoria dinámica. Cada Scope contiene un arreglo de símbolos, que puede aumentar su capacidad cuando se agregan más declaraciones. Al destruir un ámbito mediante **scope_destroy()**, se liberan los símbolos que contiene, el arreglo y el propio ámbito.

Una decisión que tomamos fue no liberar los ámbitos inmediatamente después de salir de ellos. Cuando el analizador termina de revisar un bloque, el ámbito deja de estar disponible para las búsquedas, pero se conserva en memoria hasta que se destruye la tabla de símbolos.

Para esto utilizamos **owned_scopes**, que mantiene un registro de los ámbitos creados. Al finalizar el análisis, **symbol_table_destroy()** recorre estos ámbitos y libera todos sus recursos.

También tuvimos que cuidar la relación entre la tabla de símbolos y el AST. Cada símbolo guarda una referencia al nodo donde fue declarada la variable, pero no es responsable de liberar ese nodo, ya que pertenece al árbol.Por esto, cuando acaba el programa se libera primero la información del analizador semántico mediante **semantic_analyzer_destroy()** y después el AST mediante **ast_destroy().**

Diagnósticos y recuperación

El análisis semántico conserva el manejo de errores léxicos y sintácticos de las prácticas anteriores. Si el lexer o el parser encuentran un error, el compilador muestra el diagnóstico correspondiente y no comienza el análisis semántico.

En esta práctica agregamos nuevos diagnósticos para detectar principalmente dos problemas:
1.- utilizar una variable que no ha sido declarada
2.- declarar nuevamente una variable dentro del mismo ámbito.
Para reportar los errores utilizamos la información de línea y columna que ya guardábamos en los nodos del AST.

x = 5;

Error semantico [1:0]: la variable x no ha sido declarada.

Y si declaramos dos veces una variable:

int x;
bool x;

Error semantico [2:5]: la variable 'x' ya fue declarada en este ambito. Declaracion anterior [1:4].

En este caso también mostramos la posición de la primera declaración, ya que esa información se conserva dentro del símbolo registrado en la tabla.

Una parte importante de esta práctica fue permitir que el analizador continúe revisando el AST después de encontrar un error semántico. Para realizar esto utilizamos **error_count**, que registra la cantidad de errores semánticos encontrados durante el recorrido.

También distinguimos los errores semánticos de los fallos internos. Por ejemplo, si no es posible reservar memoria para crear un símbolo o un ámbito, el analizador registra un fallo interno y detiene el recorrido para evitar utilizar estructuras incompletas.

Todos los diagnósticos se envían a stderr. Cuando existe algún error, stdout permanece vacío y el programa termina con un código distinto de cero.

Resultados y pruebas

Para comprobar el funcionamiento del analizador semántico utilizamos las pruebas públicas proporcionadas para la Práctica 5. También conservamos las pruebas del lexer como regresión y agregamos pruebas propias para revisar las nuevas estructuras y funcionalidades.

Las pruebas públicas contienen 24 casos que incluyen programas válidos e inválidos. Con ellas comprobamos declaraciones de variables, redeclaraciones, ámbitos anidados, shadowing, uso de variables no declaradas y recuperación ante errores.

También verificamos que el compilador conserve el comportamiento solicitado para está práctica principalmente que un programa válido imprima únicamente el mensaje de éxito semántico y que los errores se envíen a stderr.

Para comprobar cada módulo por separado agregamos cuatro pruebas unitarias:

Prueba	Qué es lo que verifica
symbol_unit.c	**Creación de símbolos, almacenamiento de información y liberación de memoria.**
scope_unit.c	**Creación de ámbitos, niveles y referencias al ámbito padre.**
symbol_table_unit.c	**Inserción de símbolos, búsquedas, redeclaraciones, shadowing y salida de**

	ámbitos.
semantic_unit.c	**Recorrido del AST, detección de variables no declaradas y acumulación de errores.**

Y desarrollamos 7 pruebas propias para casos adicionales:

Prueba	Qué es lo que verifica
shadowing_multinivel	**Variables con el mismo nombre en tres niveles de ámbito.**
errores_acumulados	**Detección de varios errores semánticos dentro del mismo programa.**
uso_fuera_de_ambito	**Uso de una variable después de salir del bloque donde fue declarada.**
bloques_hermanos	**Declaraciones independientes dentro de bloques diferentes.**
tipos_pospuestos_p06	**Que todavía no se compruebe la compatibilidad de tipos.**
operandos_no_declarados	**Identificadores no declarados dentro de expresiones.**
trescientas_variables	**Crecimiento dinámico de la tabla de símbolos al registrar 300 variables.**

Con la prueba de 300 variables comprobamos que los arreglos utilizados para almacenar los símbolos pueden aumentar su capacidad y que las búsquedas continúan funcionando cuando se registran muchas declaraciones.

Problemas encontrados

Durante el desarrollo de esta práctica tuvimos que considerar varios puntos para que el analizador semántico funcionara correctamente junto con el código de las prácticas anteriores.

- Uno de los principales puntos fue entender cómo funcionaban los ámbitos y las búsquedas de variables. Al principio pensamos que solamente necesitabamos guardar los nombres de las variables en una tabla, pero también tenemos que considerar dónde fueron declaradas. Para resolver esto utilizamos Scope y las

funciones de búsqueda local y visible, que nos permiten distinguir entre una redeclaración y un caso válido de shadowing.

- Otro punto importante fue el manejo de memoria. Como ahora creamos símbolos y ámbitos utilizando memoria dinámica, teníamos que asegurarnos de que todos esos recursos se liberaran correctamente. También fue necesario considerar que los símbolos guardan referencias a los nodos del AST, pero no deben liberarlos. Para manejar esto decidimos conservar los ámbitos hasta destruir la tabla de símbolos y utilizar funciones específicas para liberar cada estructura.

- También tuvimos que adaptar la Práctica 4 para integrar el análisis semántico sin modificar lo que ya funcionaba. Para esto agregamos los nuevos módulos al Makefile y modificamos main.c, de manera que el análisis semántico solamente se ejecutara después de obtener un AST válido. Además, cambiamos la salida del compilador porque ahora ya no debía imprimir el árbol ni el mensaje de éxito sintáctico.

- Finalmente, tuvimos que considerar que las pruebas de las prácticas anteriores no siempre podían utilizarse de la misma manera. Por ejemplo, las pruebas de P04 esperaban la impresión del AST, mientras que en P05 la salida es diferente. Por eso conservamos las pruebas anteriores como referencia y agregamos pruebas públicas, unitarias y propias para comprobar las nuevas funcionalidades.

División del trabajo

Para desarrollar esta práctica dividimos las actividades entre la implementación de la tabla de símbolos, el analizador semántico, las pruebas y la documentación.

Guzmán Ramirez Diego Arturo

Hizo la implementación de la tabla de símbolos y el manejo de ámbitos. Los módulos Symbol, Scope y SymbolTable, junto con las funciones para crear símbolos, registrar declaraciones, realizar búsquedas y administrar los ámbitos del programa.

También las pruebas unitarias de estos módulos y en la revisión de memoria mediante Valgrind, para comprobar que los símbolos y ámbitos se liberaran correctamente.
En la documentación, las partes del reporte relacionadas con el diseño de la tabla de símbolos, los ámbitos, la resolución de identificadores y el manejo de memoria.

Marcial Ahedo Andrick Iorel

Trabajó en el analizador semántico, incluyendo el recorrido del AST para verificar declaraciones, asignaciones, expresiones, bloques y estructuras de control. También participó en la detección de errores semánticos y en la integración de los nuevos módulos con main.c y el Makefile.
Incorporar las pruebas públicas de P05 y desarrollar las pruebas propias del analizador semántico, verificando el funcionamiento de las declaraciones, los ámbitos, el shadowing y la acumulación de errores.
En la documentación, la actualización de README.md, CHANGELOG.md y tests/README.md. También se encargó de organizar el reporte y realizar el diagrama.

Uso de herramientas de Inteligencia Artificial

Durante el desarrollo de la práctica se utilizó ChatGPT, de OpenAI, como herramienta de apoyo para cumplir todos los requisitos de la práctica, analizar errores de compilación, revisar alternativas de implementación/test propios, y apoyar la organización de la documentación.
Principalmente lo usamos en la revisión de la redacción para que se entendiera de forma clara el avance de la práctica anterior al de está práctica, queríamos que los tecnicismos se usaran de forma correcta y la explicación que dimos se conectara de forma clara.
La descripción de algunos casos de prueba y dudas específicas en la implementación de la práctica que se especifican en la sección de problemas encontrados.
Para especificar las referencias utilizadas y para organizar los README.md con el formato de markdown y no modificar la plantilla que se encuentra en el ZIP de la práctica.
También la utilizamos para revisar los lineamientos de entrega con el propósito de que todo el contenido corresponda con los requisitos solicitados.

Prompts:

“Revisa el PDF de los lineamientos y dime si nos falta alguno de ellos, te adjunto el reporte para que puedas verificarlo, en caso de que falten requisitos dime cuáles.”

“Ya que revisamos que las secciones son correctas, quiero que mejores el reporte, enfócate en la redacción, hazlo de forma formal y de manera que no pierda mis ideas pero que se entienda con claridad, es decir si se puede desarrollar más o no consideré algo en la redacción agregalo, fijate en todas las secciones, tomate tu tiempo”

“Te adjunto los README para que puedas escribirlos con el formato de markdown y se vean con claridad todos los ejemplos, sobre todo organizarlos y completarlos según la plantilla”

“Tengo la idea de implementar dos funciones para buscar variables, una que busque solamente en el ámbito actual y otra que también revise los ámbitos padres.
Pero no entiendo qué casos debería utilizar cada una y cómo evito que una variable se registre dos veces en el mismo ámbito?”

“cuando salgamos de un bloque, en lugar de liberar inmediatamente su Scope, podemos guardarlo hasta que termine el análisis y después liberarlos todos los ámbitos juntos? no sé si eso puede causar problemas con los apuntadores u otro problema “

“Ya implementamos la tabla de símbolos y el analizador semántico, pero queremos revisar que liberamos correctamente la memoria. Ayúdame a utilizar Valgrind y a interpretar sus resultados.”

“Quiero conservar el lexer, parser y AST que ya tengo hecho, pero quiero agregar el análisis semántico después de que se construya el árbol , cómo lo puedo integrar sin romper nada? o qué debería considerar primero?”

“Te adjunte el ZIP completo de la práctica y el de mi repositorio, analizalos y dime si ves alguna incoherencia en nuestra estructura del repositorio o si ves algún requerimiento faltante según las indicaciones del PDF y del material de la práctica.”

Conclusiones

En esta práctica logramos agregar el análisis semántico a nuestro compilador MiniC, utilizando el AST de la Práctica 4. Implementamos una tabla de símbolos que nos permite registrar variables, manejar ámbitos y detectar errores como variables no declaradas o redeclaraciones.

También comprobamos que los nuevos módulos funcionan correctamente mediante las pruebas realizadas y Valgrind. Con esto, nuestro compilador ya puede realizar algunas verificaciones semánticas y queda preparado para continuar con la comprobación de tipos en la siguiente práctica.

Referencias

1. Salazar González, P. Y. Compiladores — Práctica 5. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
2. Salazar González, P. Y. Definición de MiniC. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
3. Salazar González, P. Y. Lineamientos de Entrega de Prácticas. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
4. Guía para la elaboración del reporte de la Práctica 5. Material proporcionado en el curso de Compiladores, Facultad de Ciencias, UNAM, semestre 2027-1.